

Graph-Based Vulnerability Retrieval

Master's Thesis — Technical Progress · Checkpoint 3

Lívia

TU Munich · Siemens

April 29, 2026

Agenda

- 🔄 **Checkpoint 2 recap** — results & open questions
- ✂️ **Slicing ablation** — full graph vs. sliced, with/without diff labels
- 🏗️ **CWE ontology** — structure & Wu-Palmer similarity
- 🔗 **Cross-CVE callgraph** — shared callee signatures as a second signal
- 📖 **Knowledge-augmented reranking** — embedding + Jaccard fusion
- 📊 **Embedder ensembles** — majority vote, cascade, RRF strategies
- 🔴 **Confusion analysis** — where every embedder still fails
- 💡 **Key findings & next steps**

Slicing Ablation: Nomenclature & Metrics

Graph variants

Symbol	Definition
G_{before}	Full pre-patch CPG. No diff labels, no slicing.
$G_{\text{before}}^{+\ell}$	Full pre-patch CPG + diff labels transferred from G_{vuln} . Same size, extra node annotations.
$G_{\text{vuln}}^{-\ell}$	Fingerprint-sliced subgraph. Diff attributes (diff, diff_weight) stripped.
G_{vuln}	Fingerprint-sliced subgraph + diff labels + weights ($w \in \{1.0, 0.8, 0.6, 0.2\}$).

Labels: removed ($w=1.0$), fix_adjacent ($w=0.8$), edge_changed ($w=0.6$), context ($w=0.2$).

Evaluation metrics

Metric	Definition
hit@k	Fraction of queries whose correct CVE appears in top- k results.
MRR	Mean reciprocal rank of the first correct CVE.
CVE Prec.	Per-CVE binary hit (was the CVE found in top- k ?), macro-averaged across CVE classes.
CVE Recall	Fraction of same-CVE index items retrieved in top- k , macro-averaged.
CWE Recall	Fraction of same-CWE items retrieved, macro-averaged over CWE classes.

All precision/recall/F1 are **macro-averaged** per class (each CVE/CWE contributes equally regardless of frequency).

Slicing Ablation: Experiment Setup

Setup: 225 index, 73 query (seed=42, stratified split). Symmetric protocol: query uses same graph variant as index. PCA fitted on index only; queries projected through index PCA.

We compare four graph representations, progressively adding diff labels and fingerprint slicing:

Variant	Sliced	Labels	What changes
G_{before}	—	—	Full pre-patch CPG. Baseline — embedders see the entire graph.
$G_{\text{before}}^{+\ell}$	—	✓	Full pre-patch CPG + diff labels transferred from G_{vuln} . Same size, extra node annotations.
$G_{\text{vuln}}^{-\ell}$	✓	—	Fingerprint-sliced subgraph; diff attributes stripped.
G_{vuln}	✓	✓	Sliced subgraph + diff labels and weights ($w \in \{1.0, 0.8, 0.6, 0.2\}$).

Run 02006d. 6 embedders $\times$ 4 variants = 24 configurations.

Slicing Ablation: Results ($n=73$)

Embedder	G_{before} (full)			$G_{\text{before}}^{+\ell}$ (full + labels)			$G_{\text{vuln}}^{-\ell}$ (sliced, no labels)			G_{vuln} (sliced + labels)		
	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE
Combined	76.7%	.802	.405	76.7%	.802	.405	79.5%	.834	.489	79.5%	.834	.489
GIN	75.3%	.814	.518	75.3%	.814	.518	72.6%	.788	.443	72.6%	.788	.443
CodeBERT seq	75.3%	.773	.385	75.3%	.793	.412	73.9%	.770	.383	75.3%	.793	.412
CodeBERT pat.	73.9%	.764	.394	76.7%	.793	.413	72.6%	.762	.375	71.2%	.768	.415
WL	71.2%	.751	.375	71.2%	.751	.375	68.5%	.725	.385	68.5%	.725	.385
RGCN	73.9%	.767	.395	58.9%	.668	.372	72.6%	.768	.401	60.3%	.676	.372

Labels on full graph ($G_{\text{before}} \rightarrow G_{\text{before}}^{+\ell}$)

- Structural embedders: ± 0 pp (ignore text attrs)
- CodeBERT pat.: +2.8 pp (labels help)
- RGCN: -15.0 pp (labels harm)

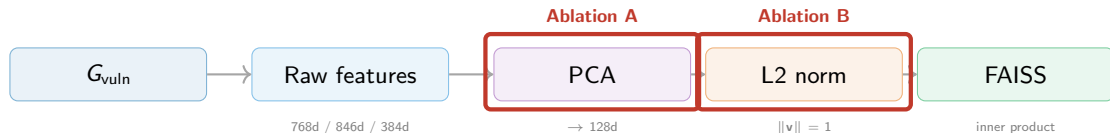
Effect of slicing ($G_{\text{before}} \rightarrow G_{\text{vuln}}^{-\ell}$)

- Combined: +2.8 pp (PCA benefits from focused subgraph)
- GIN: -2.7 pp (loses global connectivity)
- Full graphs: all ≈ 74 –76%

Labels on sliced ($G_{\text{vuln}}^{-\ell} \rightarrow G_{\text{vuln}}$)

- CodeBERT seq: +1.4 pp
- Structural: ± 0 pp
- RGCN: -12.3 pp (diff types confuse MP)

Embedding Pipeline: Where PCA & L2 Normalization Sit



PCA (dimensionality reduction)

- Projects high-dim features to 128d
- Learns optimal linear basis from data
- Decorrelates and compresses — fuses heterogeneous streams (e.g. 78d structural + 768d CodeBERT)

L2 normalization (scale invariance)

- Maps every embedding onto the unit hypersphere
- FAISS inner product = cosine similarity
- Similarity depends on **direction** (pattern), not **magnitude** (graph size)

Ablation Results: PCA and L2 Normalization ($n=225$, self-retrieval)

	PCA + L2 (baseline)			No PCA, L2 only (Abl. A)			PCA, no L2 (Abl. B)		
Embedder	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE
Combined	84.9%	.890	.501	74.0%	.780	.398	1.4%	.047	.071
CodeBERT seq	75.3%	.793	.412	76.7%	.799	.402	1.4%	.107	.160
GIN	72.6%	.788	.443	72.6%	.788	.443	1.4%	.044	.068
CodeBERT pat.	71.2%	.768	.414	75.3%	.788	.394	1.4%	.132	.201
WL	65.8%	.720	.423	67.1%	.723	.435	0.0%	.045	.065
RGCN	60.3%	.676	.372	67.1%	.733	.390	0.0%	.014	.060

Baseline: PCA + L2

- Combined dominates: 84.9%
- PCA is critical for Combined
(−10.9 pp without)

Abl. A: No PCA

- CodeBERT stable (± 1 pp)
- GIN identical — already 128d
- Combined collapses: PCA fuses the 3 sub-embedders

Abl. B: No L2

- **Catastrophic failure** — all $< 2\%$
- Mean sim $> 100k$ (magnitude dominates)
- L2 norm is non-negotiable

Why L2 Normalization Is Non-Negotiable

The magnitude problem

Without L2 normalization, FAISS inner product computes:

$$\text{score}(q, d) = \mathbf{q} \cdot \mathbf{d} = \|\mathbf{q}\| \|\mathbf{d}\| \cos \theta$$

The score is dominated by $\|\mathbf{q}\| \|\mathbf{d}\|$, not $\cos \theta$.

Mean pairwise similarity

Embedder	With L2	No L2
Combined	0.02	416,081
CodeBERT seq	0.01	456
GIN	0.99	260,029
RGCN	0.14	4,281

Without L2, all vectors appear “close” — a graph with more nodes produces a larger vector that dominates inner-product ranking.

With L2: direction-only similarity

$$\text{score}(q, d) = \cos \theta = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \in [-1, 1]$$

- Every vector has $\|\mathbf{v}\| = 1$
- Inner product $\equiv$ cosine similarity
- Similarity measures **what kind** of vulnerability, not **how much code** changed
- A 20-node UAF patch and a 200-node UAF patch can still be nearest neighbours

Key takeaway

L2 normalization is **essential** for the pipeline. Removing it causes complete retrieval failure across all 6 embedders. PCA matters mainly for multi-stream embedders (Combined: -11 pp without it).

Agent Baseline: Experiment Design

Goal

Given a vulnerable function and a retrieved similar CVE, can an LLM generate a correct patch **by analogy**?

Compare *oracle* retrieval (perfect) vs. *embedding* retrieval (our pipeline).

Model & prompt

- **GPT-4o** (Azure), temperature=0.2, max_tokens=4096
- **One-shot patching by analogy**: system prompt contains the retrieved CVE's fix-list, variable/function mapping, root cause
- Assistant message shows the example's CoT + patched code
- User message provides the target function to patch
- Agent must output [CoT START]...[Patched Code END]

Retrieval modes

Mode	What it retrieves
Oracle	Same-CVE pair from index (perfect match) — upper bound on agent quality
Embedding	Top-1 from Combined embedder + FAISS (realistic pipeline)

Evaluation protocol

- Same split: 225 index, 73 query (seed=42)
- **Patch quality**: BLEU-4, token Jaccard, CodeBLEU proxy
- **Retrieval**: hit@1, MRR, CWE hit@5
- **Confidence**: precision/recall at score thresholds

Agent Baseline: Retrieval & Patch Quality

Retrieval feeding the agent ($n=73$)

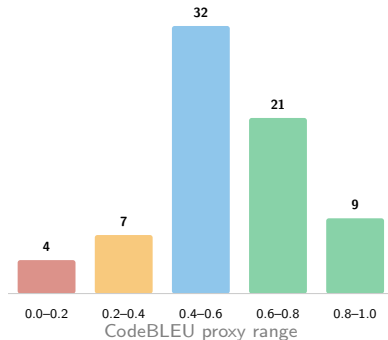
Same Combined embedder from §2 — Oracle hit@1 86.3%, Embedding 68.5%, both hit@5 $\geq 80\%$. The 17.8 pp gap is the “analogy quality” difference between modes.

Patch quality metrics

Metric	Oracle $n=73$	Embedding $n=73$
CodeBLEU proxy	.557	.409
BLEU-4	.542	.388
Token Jaccard	.585	.461
Char seq. ratio	.506	.373
Exact match	0	0

Key finding: Retrieval quality **matters** — CodeBLEU drops from .557 (oracle) to .409 (embedding), a -15 pp gap. The 17.8 pp retrieval gap translates into measurable patch quality loss.

CodeBLEU proxy distribution (Oracle, $n=73$)



By CWE class (oracle, best/worst)

CWE class	CBP
Use of Uninit. Variable ($n=2$)	.818
Memory Leak ($n=2$)	.661
Integer Overflow ($n=10$)	.637

Agent Baseline: Analysis & Failure Modes

Where the agent succeeds

- **Mechanical fixes:** null-pointer guards, missing return-value checks, uninitialised variables — high token overlap (CBP > 0.6)
- **Pattern-transferable CWEs:** Integer overflow, Deadlock, Memory leak — fix patterns are regular and the fix-list maps cleanly
- 41% of patches score CBP > 0.6 (oracle), 30% (embedding); the agent captures the *structure* of the fix even when exact tokens differ

Where the agent fails

- **Zero exact matches:** the LLM adds comments, reformats whitespace, and paraphrases variable names — syntactically different but potentially semantically correct
- **Race conditions** (CBP .424): fixes require inserting lock/unlock pairs at precise scope boundaries — hard to transfer by analogy
- **Complex structural rewrites** (Resource Lifetime CBP .025): fix involves rearchitecting control flow, not just inserting a guard

Oracle vs. Embedding: what the gap tells us

- Patch quality **degrades with retrieval accuracy:** CBP .557 (oracle) vs. .409 (embedding), -15 pp
- When retrieval misses the correct CVE (31.5% of queries), the analogy is less relevant → lower patch quality
- Both retrieval **and** generation quality are bottlenecks

Open questions for improvement

- **Better evaluation:** CodeBLEU proxy is a surface metric — does a CBP of 0.5 mean “half correct” or “correct but reformatted”? Compilation + test-based evaluation needed
- **Multi-shot:** retrieve top-*k* examples instead of one
- **Iterative refinement:** let the agent self-critique and revise
- **Larger context:** include caller/callee code, not just the function

Takeaway

Retrieval quality **directly impacts** patch quality — the 17.8 pp retrieval gap causes a 15 pp CBP loss. Improving both retriever and agent is needed.

Thank you

Repository: `graph-rag/` `python experiments/knowledge_experiment.py`